

# TransitOps

Architecture, Data Model, API, and UX Design

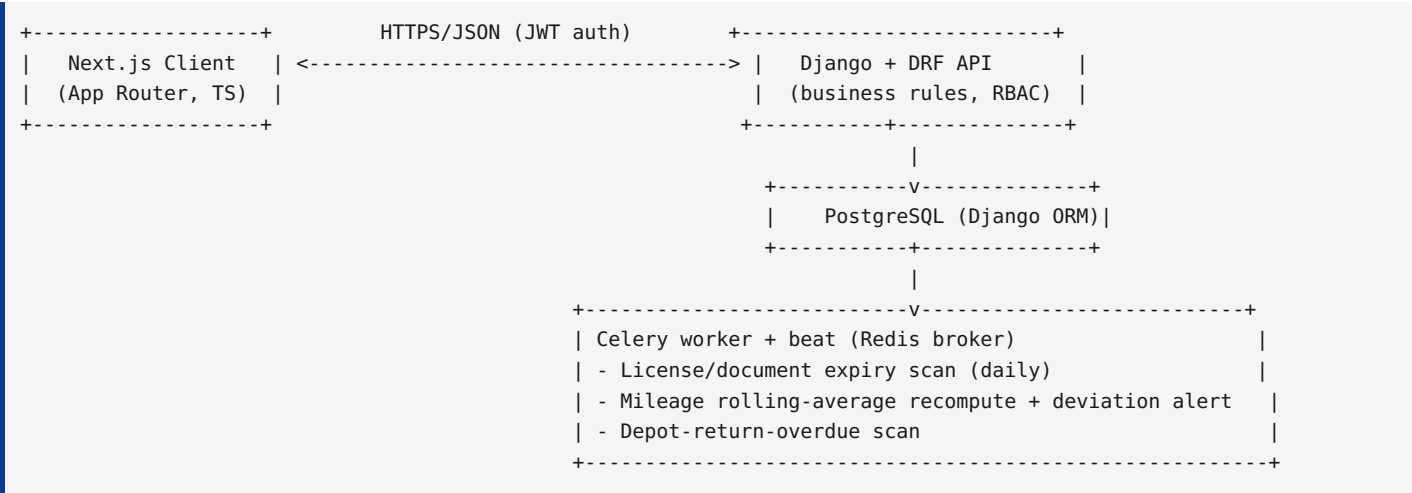
TransitOps Team

July 2026 · v1.0

- 1. Architecture Overview
- 2. Module → Screen Map
- 3. Data Model (ER Overview)
  - 3.1 Core Entities
  - 3.2 Key Relationships
  - 3.3 Derived / Computed Fields (not stored, computed at read time or via signal)
- 4. API Design (REST, DRF)
- 5. RBAC Matrix
- 6. State Machines
  - 6.1 Trip Lifecycle
  - 6.2 Vehicle Status
  - 6.3 Driver Status
  - 6.4 Maintenance Status
  - 6.5 Tyre Lifecycle (*stretch*)
- 7. Business Rule Enforcement Design
- 8. UX / Screen Design Notes
- 9. Supporting Design Details
  - 9.1 Mileage Deviation Alert — Calculation
  - 9.2 Vehicle ROI Formula
  - 9.3 Fallback for Background Jobs
- 10. Security Design

# 1. Architecture Overview

TransitOps is a decoupled client-server web application:



**Why this shape:** - Next.js handles all rendering/UX and calls the API purely over REST — keeps frontend and backend independently deployable/testable, important when a hackathon team splits frontend/backend work in parallel. - Django + DRF centralizes **all** business-rule enforcement server-side (never trust client-side validation alone) — critical since the PS explicitly grades on business rule correctness. - Celery is isolated to the “smart” features (reminders, anomaly detection) so the core CRUD/trip flow has zero dependency on it — if Celery/Redis setup runs out of time, it can be dropped without breaking the Must-have MVP (the same logic can run synchronously as a fallback, see §9.3).

# 2. Module → Screen Map

| #  | Module                    | Primary Role(s)                  | Wireframe Screen                      |
|----|---------------------------|----------------------------------|---------------------------------------|
| 0  | Authentication (RBAC)     | All                              | Login screen                          |
| 1  | Dashboard                 | All (scoped view)                | Dashboard                             |
| 2  | Vehicle Registry          | Fleet Manager                    | Vehicle Registry                      |
| 3  | Driver Management         | Safety Officer, Fleet Manager    | Drivers & Safety Profiles             |
| 4  | Trip Dispatcher           | Dispatcher                       | Trip Dispatcher                       |
| 5  | Maintenance               | Fleet Manager                    | Maintenance                           |
| 6  | Fuel & Expense            | Financial Analyst                | Fuel & Expense Management             |
| 7  | Reports & Analytics       | Financial Analyst, Fleet Manager | Reports & Analytics                   |
| 8  | Settings & RBAC           | Admin/Fleet Manager              | Settings & RBAC                       |
| 9  | Vehicle Documents (enh.)  | Fleet Manager, Safety Officer    | New sub-panel inside Vehicle Registry |
| 10 | Tyre Management (stretch) | Fleet Manager                    | New sub-module inside Maintenance     |

# 3. Data Model (ER Overview)

## 3.1 Core Entities

**User** - id, email (unique), password\_hash, full\_name, role\_id (FK Role), depot\_id, is\_active, last\_login

**Role** - id, name (Fleet Manager / Dispatcher / Safety Officer / Financial Analyst), permission\_matrix (JSON or via M2M Permission table)

**Vehicle** - id, registration\_number (unique), name\_model, type (Van/Truck/Mini/etc.), max\_load\_capacity\_kg, odometer\_km, acquisition\_cost, status (Available/On Trip/In Shop/Retired), region, created\_at

**VehicleDocument** (*enhancement*) - id, vehicle\_id (FK), doc\_type (RC/Insurance/PUC/Permit), doc\_number, issue\_date, expiry\_date, file\_attachment\_url (optional)

**Driver** - id, name, license\_number, license\_category, license\_expiry\_date, contact\_number, safety\_score, status (Available/On Trip/Off Duty/Suspended)

**Trip** - id, trip\_code, source, destination, vehicle\_id (FK, nullable until assigned), driver\_id (FK, nullable), cargo\_weight\_kg, planned\_distance\_km, load\_type (enh.), freight\_type (enh.), status (Draft/Dispatched/Completed/Cancelled), dispatched\_at, completed\_at, final\_odometer\_km, fuel\_consumed\_l, home\_depot\_id (enh.), created\_by

**FuelLog** (Trip Expense — fuel) - id, vehicle\_id (FK), trip\_id (FK, nullable), date, litres, cost, odometer\_at\_fill, computed\_mileage\_kmpl (derived), is\_anomaly\_flagged (bool, enh.)

**Expense** (Toll/Misc/Other) - id, vehicle\_id (FK), trip\_id (FK, nullable), category (Toll/Misc/Other), amount, date, notes

**MaintenanceRecord** - id, vehicle\_id (FK), service\_type, cost, date, status (In Shop/Completed), closed\_at

**Tyre** (*stretch*) - id, vehicle\_id (FK), serial\_number, purchase\_date, current\_position (enum), cumulative\_km, status (Active/Retired/Worn)

**TyrePositionHistory** (*stretch*) - id, tyre\_id (FK), position, odometer\_at\_change, date, km\_run\_in\_position (derived on close-out)

**Notification** (*enhancement*) - id, type (License Expiry / Document Expiry / Mileage Deviation / Depot Overdue), reference\_entity, reference\_id, message, is\_read, created\_at

### 3.2 Key Relationships

- One Vehicle → many FuelLogs, Expenses, MaintenanceRecords, VehicleDocuments, Tyres
- One Trip → one Vehicle, one Driver (at a time), many Expenses, one auto-generated FuelLog on completion
- One Tyre → many TyrePositionHistory rows (append-only ledger, not overwritten, so km-per-position is fully reconstructable)

### 3.3 Derived / Computed Fields (not stored, computed at read time or via signal)

- `Vehicle.total_operational_cost` = `SUM(FuelLog.cost)` + `SUM(MaintenanceRecord.cost)` [+ `SUM(Expense.amount)`]
- `Vehicle.fuel_efficiency_kmpl` = Distance / Fuel over a period
- `Vehicle.roi` = (Revenue – (Maintenance + Fuel)) / Acquisition Cost
- `Vehicle.rolling_avg_mileage` = average of `computed_mileage_kmpl` over last N fuel logs
- `Fleet.utilization_pct` = Active Trips vehicles / Total non-retired vehicles

## 4. API Design (REST, DRF)

Base path: `/api/v1/`

| Resource | Endpoints                                                                                        | Roles (write) | Roles (read) |
|----------|--------------------------------------------------------------------------------------------------|---------------|--------------|
| Auth     | POST <code>/auth/login</code> , POST <code>/auth/refresh</code> , POST <code>/auth/logout</code> | —             | All          |

|                          |                                                                                                                                        |                                                       |                               |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|-------------------------------|
| Vehicles                 | GET/POST /vehicles/,<br>GET/PATCH/DELETE<br>/vehicles/{id}/                                                                            | Fleet Manager                                         | All                           |
| Vehicle Documents        | GET/POST<br>/vehicles/{id}/documents/                                                                                                  | Fleet Manager                                         | Fleet Manager, Safety Officer |
| Drivers                  | GET/POST /drivers/,<br>GET/PATCH/DELETE<br>/drivers/{id}/                                                                              | Safety Officer, Fleet Manager                         | All                           |
| Trips                    | GET/POST /trips/, GET/PATCH<br>/trips/{id}/, POST<br>/trips/{id}/dispatch/, POST<br>/trips/{id}/complete/, POST<br>/trips/{id}/cancel/ | Dispatcher                                            | All                           |
| Maintenance              | GET/POST /maintenance/,<br>PATCH<br>/maintenance/{id}/close/                                                                           | Fleet Manager                                         | All                           |
| Tyres ( <i>stretch</i> ) | GET/POST<br>/vehicles/{id}/tyres/, POST<br>/tyres/{id}/reposition/                                                                     | Fleet Manager                                         | Fleet Manager                 |
| Fuel Logs                | GET/POST /fuel-logs/                                                                                                                   | Financial Analyst, Dispatcher (at<br>trip completion) | All                           |
| Expenses                 | GET/POST /expenses/                                                                                                                    | Financial Analyst                                     | All                           |
| Reports                  | GET /reports/dashboard/, GET<br>/reports/analytics/, GET<br>/reports/export.csv                                                        | —                                                     | Role-scoped                   |
| Notifications            | GET /notifications/, PATCH<br>/notifications/{id}/read/                                                                                | System-generated                                      | All                           |
| Settings/RBAC            | GET/PATCH /settings/,<br>GET/PATCH<br>/roles/{id}/permissions/                                                                         | Fleet Manager (admin)                                 | All (read)                    |

All write endpoints validate business rules server-side inside a DRF serializer `validate()` or a service-layer function called from the view — never solely via DB constraints, so validation errors return clean messages (matching the wireframe’s inline error banners, e.g. “Capacity exceeded by 200kg — dispatch blocked”).

## 5. RBAC Matrix

| Module                   | Fleet Manager | Dispatcher | Safety Officer     | Financial Analyst |
|--------------------------|---------------|------------|--------------------|-------------------|
| Dashboard                | ✓             | ✓          | ✓                  | ✓                 |
| Fleet / Vehicle Registry | ✓ (full)      | view       | —                  | view              |
| Drivers                  | ✓ (full)      | view       | ✓ (full)           | —                 |
| Trips / Dispatch         | view          | ✓ (full)   | —                  | view              |
| Maintenance              | ✓ (full)      | —          | —                  | view              |
| Fuel & Expenses          | —             | log only   | —                  | ✓ (full)          |
| Analytics/Reports        | ✓             | —          | ✓ (safety metrics) | ✓                 |
| Settings & RBAC          | ✓ (admin)     | —          | —                  | —                 |

This matches the Settings screen in the wireframe (checkmarks/dashes per role per module) — implement as a `RolePermission` table so it’s editable, not hardcoded.

## 6. State Machines

---

### 6.1 Trip Lifecycle

---

Draft → Dispatched → Completed Draft → Cancelled (Or) Dispatched → Cancelled

Guard conditions: - Draft → Dispatched: vehicle & driver both Available,  $\text{cargo\_weight} \leq \text{vehicle.max\_load\_capacity}$ , driver license valid & not Suspended - Dispatched → Completed: requires `final_odometer_km` and `fuel_consumed_l` input - Dispatched → Cancelled: restores vehicle & driver to Available

### 6.2 Vehicle Status

---

Available ⇌ On Trip (via trip dispatch/complete/cancel) Available ⇌ In Shop (via maintenance open/close) Available/In Shop → Retired (manual, terminal — Retired never re-enters Available automatically)

### 6.3 Driver Status

---

Available ⇌ On Trip (via trip dispatch/complete/cancel) Available ⇌ Off Duty (manual toggle) Any → Suspended (Safety Officer action; blocks trip assignment until lifted)

### 6.4 Maintenance Status

---

In Shop → Completed (closing triggers vehicle status restore unless Retired)

### 6.5 Tyre Lifecycle (*stretch*)

---

Active (position A) → Active (position B) via reposition event (closes km ledger on position A, opens on B) Active → Retired/Worn when `cumulative_km` crosses configured threshold or manually marked worn

## 7. Business Rule Enforcement Design

---

All rules from PRD §7 are implemented as a **service layer** (`services/trip_service.py`, `services/vehicle_service.py`, etc.) sitting between DRF views and the ORM — views never mutate model state directly. Each service function:

1. Loads the relevant objects inside a DB transaction (`select_for_update()` on Vehicle/Driver rows to prevent race conditions on double-dispatch).
2. Validates preconditions (status, capacity, license expiry) and raises a domain-specific exception with a user-facing message on failure.
3. Performs the state transition + any derived writes (e.g., dispatch creates status change + audit log row) atomically.

This guarantees the “double-booking” and “overload” rules can’t be bypassed even under concurrent requests — a common hackathon-demo failure mode.

## 8. UX / Screen Design Notes

---

Design follows the provided Excalidraw wireframe closely; notes below cover additions:

- **Vehicle Registry:** add a “Documents” tab per vehicle row (RC/Insurance/PUC/Permit) with color-coded expiry badges (green >30 days, amber ≤30 days, red expired) — mirrors the status badge style already used for vehicle/driver status.
- **Trip Dispatcher:** add Load Type and Freight Type dropdowns beside Cargo Weight in the “Create Trip” form; keep the existing capacity validation banner pattern for consistency.

- **Vehicle Detail (new):** a tabbed profile page — Overview | Documents | Expenses (category-wise chart) | Trip History | Tyres (stretch) — this is the natural home for the “vehicle’s overall performance” view you described.
- **Maintenance:** add a “Tyres” sub-tab with either (a) a simple table (position, serial no, km run, last changed) for MVP, or (b) the 2D drag-drop truck diagram if time allows — both read/write the same Tyre + TyrePositionHistory data, so the simple version is a safe fallback that doesn’t block the stretch goal.
- **Dashboard:** add a “Reminders” panel (bottom-right or a bell icon with dropdown) surfacing: documents expiring soon, mileage deviation alerts, depot-return-overdue vehicles — keeps the KPI tiles clean while surfacing the “smart” features prominently for the demo.

## 9. Supporting Design Details

---

### 9.1 Mileage Deviation Alert — Calculation

---

```
mileage_this_fill = (odometer_at_fill - previous_odometer) / litres
rolling_avg = average(last N computed_mileage_kmpl for this vehicle) # N = 5, configurable
if mileage_this_fill < rolling_avg * (1 - threshold): # threshold e.g. 0.15
    flag FuelLog.is_anomaly_flagged = True
    create Notification(type=Mileage Deviation, vehicle, message)
```

### 9.2 Vehicle ROI Formula

---

```
ROI = (Revenue - (Maintenance_cost + Fuel_cost)) / Acquisition_cost
```

Revenue is entered/estimated per trip (freight revenue field) or aggregated per period for the Reports screen — documented as a manual input field on Trip if no billing module exists in MVP scope.

### 9.3 Fallback for Background Jobs

---

If Celery/Redis setup is skipped to save time, the same expiry-scan and mileage-recompute logic can run as a Django management command triggered on dashboard load (synchronous, cached for 5 minutes) — functionally identical for demo purposes, just not truly “scheduled.”

## 10. Security Design

---

- Passwords hashed via Django’s default PBKDF2 (or Argon2 if `django-argon2` is added)
- JWT access token short-lived (15 min), refresh token longer-lived (7 days), via `django-rest-framework-simplejwt`
- All endpoints require authentication except `/auth/login`
- Permission classes per view enforce the RBAC matrix (custom `IsRole('Fleet Manager')` style permission classes)
- CORS restricted to the Next.js frontend origin only
- Audit fields (`created_by`, `updated_by`, timestamps) on all mutation-heavy models (Trip, MaintenanceRecord, Vehicle status changes)